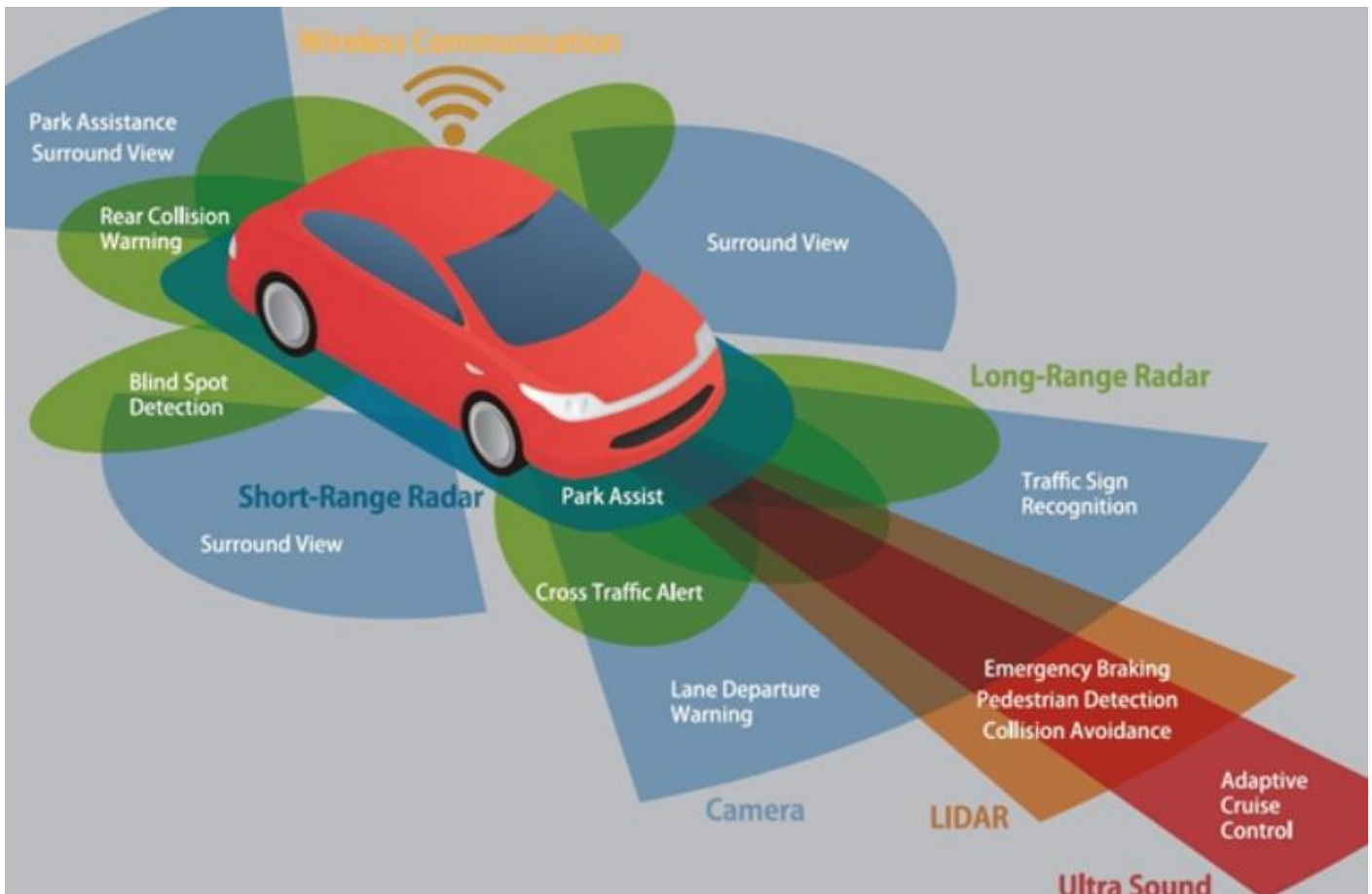


"라이다(LiDAR) 센서에 대한 이해를 돕기 위해

원리, 동작 방식, 실제 활용, 자율주행에서의 역할을 단계별로 정리"



1. LiDAR란?

- Light Detection And Ranging의 줄임말
- 레이저 빛을 쏘아 물체에 반사되어 돌아오는 시간(Time of Flight, ToF)*을 측정해 거리를 계산하는 센서다.
- 쉽게 말해, "레이저 자로 1초에 수십만 번씩 거리를 재서 주변의 3D 지도를 만드는 장치"임.

2. 동작 원리 <https://m.blog.naver.com/jhongban/223174868843>

- 1) 레이저 발사 : 수직·수평으로 여러 개의 레이저 펄스를 발사한다.
- 2) 반사 신호 수신 : 물체 표면에 맞고 돌아오는 반사광을 수광기(포토다이오드)가 감지한다.
- 3) 시간 측정
 - 빛이 나가서 돌아오기까지 걸린 시간 Δt 을 측정.
 - 거리 = (빛의 속도 * Δt) / 2 로 계산. (왕복이므로 2로 나눔)
- 4) 포인트 클라우드 생성 : 수많은 레이저 점들의 좌표(x,y,z)를 모아 3차원 점군(point cloud) 데이터를 형성한다.

3. 라이다의 특징

- 정밀한 거리 측정 (센티미터 수준 오차)
- 주간/야간 모두 사용 가능 (빛 자체를 발사하므로)
- 360° 시야 확보 가능 (회전식 LiDAR의 경우)

- 단점: 가격이 비쌈 (수천~수만 달러 → 최근은 저가형도 나옴), 비·눈·안개 같은 기상 조건에 민감, 데이터량이 방대하여 실시간 처리 필요

4. 자율주행에서의 역할

- 주행 환경 인식 : 도로 구조, 차선, 보행자, 장애물까지 3차원으로 "지도화"
- SLAM (Simultaneous Localization And Mapping) : 차량이 어디에 있는지 위치를 동시에 추정하고 주변 환경을 지도화
- 충돌 방지 : 전방의 물체 거리와 형상을 정확하게 인식

5. 라이다 vs 다른 센서

- 카메라: 색상·텍스처 같은 시각적 정보 강점, 하지만 거리 추정이 어려움
- 레이더(Radar): 장거리/악천후 강점, 하지만 해상도가 낮음
- 라이다(LiDAR): 정밀한 거리·형상 인식 강점, 하지만 비·눈에 약하고 고가

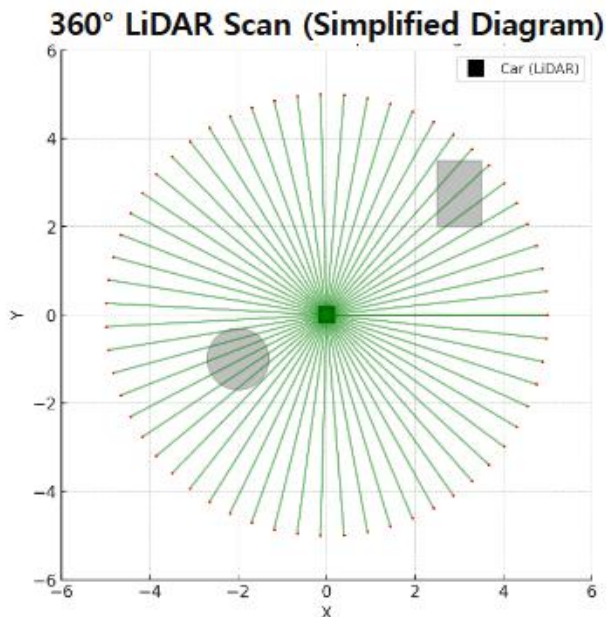
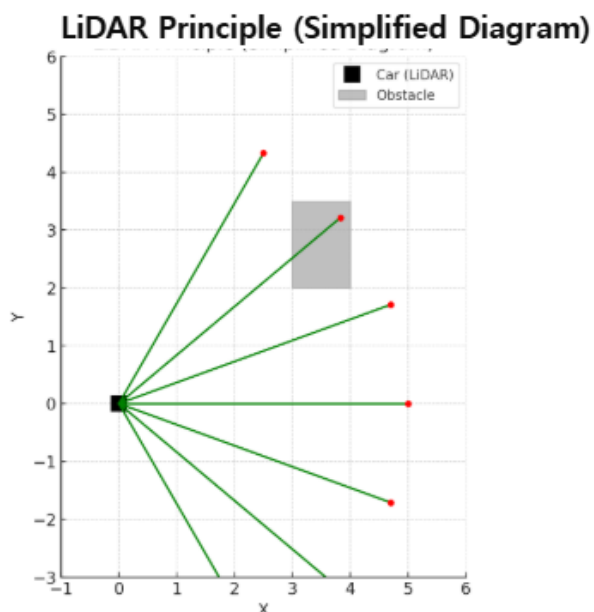
	카메라	밀리파 레이더	LiDAR (라이다)
방식	카메라에 찍힌 영상에서 물체를 식별	전파를 사용하여, 대상물에 닿아 되돌아 올 때까지의 시간차를 계측하여, 거리나 방향을 측정	레이저 광을 펄스 상태로 비추어, 대상물에 닿아 되돌아 올 때까지의 시간차를 계측하여, 거리나 위치, 형상을 3차원으로 측정
장점	• 촬영한 영상을 화상 처리함으로써 대상물을 식별할 수 있다.	• 야간이나 악천후 환경에서도 장애물 등의 방향과 거리를 계측할 수 있다. • LiDAR 대비 저렴하다.	• 거리, 위치, 형상을 고정밀도로 검출한다. • 레이저 광을 다양한 방향으로 조사(照射 / 走査)하여, 넓은 범위의 상황을 검출한다.
단점	• 정확한 형상 및 위치 검출 곤란 • 악천후 및 야간(어두운 곳), 역광에는 부적합	• 작은 물체 검출 곤란 • 골판지 상자 등 반사율이 낮은 물체 검출 곤란	• 악천후에서의 검출 능력 저하 • 밀리파 레이더 대비 비싸다.

그래서 실제 자율주행차는 카메라 + 레이더 + 라이다를 센서 융합(sensor fusion)으로 사용한다.

이해를 위해 이렇게 생각해 보자. 어두운 방에서 손전등을 비추면, 벽까지의 거리를 감으로 알 수 있다. 라이다는 손전등을 초고속으로 수십만 번 "스캐닝"해서, 방 전체의 구조를 3D로 그려내는 것과 같다.

정리하자면 라이다는 "레이저 자"를 초고속으로 수십만 번 휘둘러 정밀한 3D 지도를 만드는 센서다.

자율주행에서 주변 인식과 안전 주行的 핵심 역할을 하고, 카메라/레이더와 함께 사용되어 단점을 보완한다.



보이는 그림이 라이다 센서 원리를 단순화한 다이어그램이다.

- 검은색 네모 = 차량 위치 (라이다 센서 장착점)
- 초록색 선 = 라이다에서 발사한 빔(레이저 펄스)
- 빨간 점 = 물체에 맞아 돌아온 반사 지점
- 회색 블록 = 도로 위 장애물/벽

즉, 라이다는 차량에서 여러 방향으로 빔을 쏘고, 돌아오는 거리값을 모아서 주변 환경의 3D 지도를 그린다. 실제 차량용 라이다는 이렇게 수십~수백 개 빔을 동시에 발사해 360° 포인트 클라우드를 얻는다고 보면 된다.

""""

아래 코드는 강화학습에서 흔히 쓰이는 LiDAR 센서 개념을 아주 단순화해서 보여주는 시뮬레이터이다.

코드의 전체 목적

실제 자율주행 로봇이나 자동차는 LiDAR 센서를 사용해 주변 환경을 스캔 한다. LiDAR는 "레이저 빔을 여러 방향으로 쏘고, 부딪혀 돌아오는 시간(거리)"를 측정해서 장애물과 벽까지의 거리 분포를 알아낸다. 이 코드에서는 진짜 레이저 물리학은 아니고, 2D 평면에서 "가상의 레이(ray)를 여러 방향으로 전진시키며 충돌 여부를 체크하는 방식"으로 LiDAR를 흉내 낸 것이다. 즉, 이 코드는 라이다(LiDAR)가 어떻게 "에이전트 기준 여러 방향 거리 센서"로 동작하는지, 그리고 그것을 강화학습에서 어떻게 상태(state)로 사용할 수 있는지를 보여주는 아주 간단한 데모다.

* 주요 구성 요소

- 1) 환경(Environment)
 - 크기: WORLD_W x WORLD_H 직사각형 월드
 - 장애물: 원형(OBSTACLES) 몇 개
 - 벽: 경계선
- 2) 에이전트(Agent)
 - (x, y, theta) = 위치(x, y)와 바라보는 방향(theta)
 - 시야각(FOV)과 레이 개수(NUM_RAYS)로 LiDAR 스캔
- 3) LiDAR 시뮬레이션(cast_lidar)
 - 에이전트 방향 기준으로 여러 개의 레이(광선)를 발사
 - 한 레이(광선)는 조금씩(step 단위) 전진하다가 장애물(원)과 충돌하거나 월드 밖으로 나가면 그 지점까지의 거리를 기록
 - 모든 레이의 거리를 모아 obs라는 거리 벡터를 반환 → 강화학습에서는 이 obs가 곧 상태(state)가 됨
- 4) 시각화(plot_world)
 - matplotlib으로 월드, 장애물, 에이전트, 레이들을 그림
 - 눈으로 LiDAR 동작을 확인할 수 있음
- 5) 실행부(main)
 - cast_lidar()를 호출해 거리 관측 값을 얻고 그 결과를 출력하고 그림으로 보여줌

* 이 코드가 설명하는 것

- LiDAR는 "주변까지의 거리 벡터"를 만들어낸다. 에이전트의 눈 역할
- 강화학습에서는 LiDAR 관측 값을 상태(state)로 삼아 정책을 학습
예: obs = [1.2, 3.5, 0.7, ...] → "왼쪽 가까운 곳에 장애물이 있네? 오른쪽으로 steering!"
- LiDAR 덕분에 환경이 복잡해도 로봇은 "눈 감고 직접 보지 않고도" 경로를 찾을 수 있음

"""

실습 코드 예제)

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# ----- 환경/에이전트 설정 -----
```

```
# 시뮬레이션 세계 즉, LiDAR 센서가 탐지하는 2D 환경의 공간 경계(지도 크기)를 설정.
```

```
WORLD_W, WORLD_H = 20.0, 15.0 # 세계 크기 (가로 20, 세로 15)
```

```
WALL_THICK = 0.5 # 벽 두께(단순히 경계 밖이면 충돌로 처리). 경계판정이나 시각화 개선 시 활용 가능한 변수
```

```
# 원형 장애물 (중심 x, 중심 y, 반지름 r)
```

```
OBSTACLES = [
```

```
    (6.0, 4.0, 1.0), # (6,4) 위치 반지름 1.0 장애물
```

```
    (12.0, 10.0, 1.5), # (12,10) 위치 반지름 1.5 장애물
```

```
    (15.0, 5.0, 1.0), # (15,5) 위치 반지름 1.0 장애물
```

```
]
```

```
# 에이전트 초기 상태 (x, y, 바라보는 각도 rad 단위)
```

```
# LiDAR 시뮬레이션의 에이전트(센서가 달린 로봇)의 초기 위치와 방향(heading) 을 설정하는 부분
```

```
agent = dict(x=3.0, y=3.0, theta=np.deg2rad(30)) # 30° 방향을 바라봄
```

```
# LiDAR 파라미터
```

```
NUM_RAYS = 32 # 레이(광선) 개수
```

```
FOV = np.deg2rad(180) # 시야각 180도 (도(degree)"를 라디안(radian) 으로 변환)
```

```
# 삼각함수 등에서 각도를 다룰 때, radian은 수학에서 표준 단위, degree는 사람이 직관적으로 쓰는 단위.
```

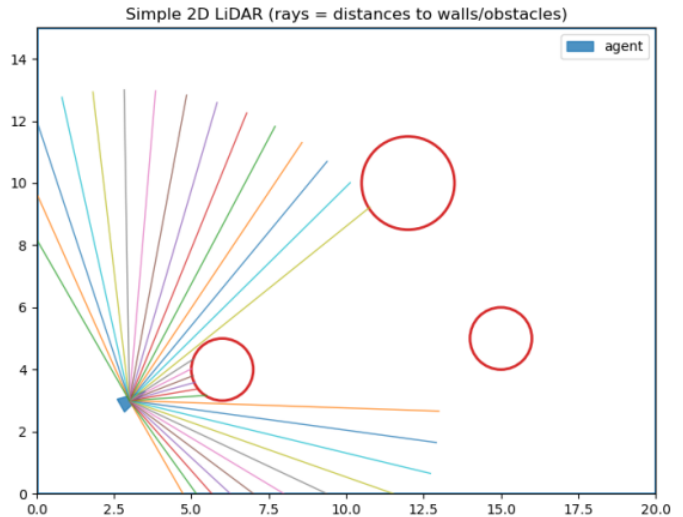
```
# 이를 변환하려면 관계식을 사용 : radians = degrees × (π / 180)
```

```
MAX_RANGE = 10.0 # 탐지 최대 거리
```

```
STEP = 0.05
```

```
# 레이 전진 단위 거리 (세밀할수록 정확 ↑). 레이가 0.05m씩 전진하며 충돌 여부 검사
```

```
# 한 레이가 최대 거리 10.0까지 나아가려면 10.0 / 0.05 = 200번의 충돌 체크를 수행
```



def inside_world(x, y):

(x,y)가 세계 경계 안에 있는지 검사

(x, y) 좌표가 $0 \leq x \leq 20$, $0 \leq y \leq 15$ 범위 안에 있으면 벽안쪽 그 밖이면 충돌(벽에 부딪힘)으로 처리.

return $(0.0 \leq x \leq \text{WORLD_W})$ and $(0.0 \leq y \leq \text{WORLD_H})$

def hit_circle(px, py, cx, cy, r):

점(px,py)이 원(cx,cy,r) 안에 있는지 검사 → 충돌 여부

점(px, py)이 원 중심(cx, cy) 반지름 r인 장애물 내부 또는 테두리 위에 있으면 True (충돌),

그렇지 않으면 False (비충돌) 를 반환하는 함수

return $(px - cx)^2 + (py - cy)^2 \leq r^2$

에이전트 (x,y,θ)에서 시야각 FOV로 NUM_RAYS개의 광선을 쏘서, 각 레이가 처음 부딪히는 지점까지의 거리를 구함.

입력: 시작점(x,y), 바라보는 각 theta, 레이 개수 num_rays, 시야각 fov, 최대거리 max_range, step.

출력: dists — 각 레이의 충돌 거리(충돌 없으면 max_range), angles — 각 레이의 절대 각도(rad)

def cast_lidar(x, y, theta, num_rays=NUM_RAYS, fov=FOV, max_range=MAX_RANGE, step=STEP):

(x,y,theta) 위치에서 num_rays개의 레이를 쏘고, 각 레이의 충돌까지 거리를 반환.

충돌이 없으면 max_range 반환

start = theta - fov/2 # 첫 번째 레이 시작 각도 (시야의 왼쪽 끝 각도)

angles는 start → start+fov까지 균등 분배된 절대각. max(num_rays-1,1)로 1개일 때도 안전.

양 끝 포함 분포(좌/우 끝 모두 레이 존재).

angles = start + np.arange(num_rays) * (fov / max(num_rays - 1, 1)) # 균등 분배된 레이 각도 배열

dists = np.full(num_rays, max_range, dtype=float) # 거리 배열 초기화: 전부 max_range

for i, ang in enumerate(angles): # 각 레이에 대해

dist = 0.0

hit = False

while dist < max_range: # 최대 거리까지 전진

px = x + np.cos(ang) * dist # 레이 끝점 X 좌표

py = y + np.sin(ang) * dist # 레이 끝점 Y 좌표

현재 레이 각도 ang로, dist를 step만큼 증가시키며 끝점 (px,py) 이동.

if not inside_world(px, py): # 벽 충돌 체크. 월드 밖 → 벽에 부딪힘

hit = True # 경계 밖이면 벽에 충돌로 처리하고 종료.

break

for (cx, cy, r) in OBSTACLES: # 모든 원형 장애물 검사

if hit_circle(px, py, cx, cy, r):

hit = True # 하나라도 원 내부에 들어가면 충돌.

break

```

        if hit:          # 충돌이면 종료
            break
        dist += step     # 충돌 없으면 한 걸음 전진

    dists[i] = min(dist, max_range)    # 충돌 거리 기록 (없으면 max_range)

return dists, angles    # 레이별 거리와 각도(절대각) 반환.

# 2D LiDAR(라이다) 환경을 시각화
# 입력값 - agent: 로봇(에이전트)의 상태 (x, y, theta 포함된 dict)
#         - rays_endpoints: 각 ray의 시작점과 끝점 좌표 리스트 → (시각화용)
def plot_world(agent, rays_endpoints=None):
    fig, ax = plt.subplots(figsize=(7.5, 5.5))
    ax.set_xlim(0, WORLD_W)    # X축 범위
    ax.set_ylim(0, WORLD_H)    # Y축 범위
    ax.set_aspect('equal', adjustable='box') # 가로 세로 비율을 1:1로 유지 (왜곡 방지)
    ax.set_title("Simple 2D LiDAR")

    # 월드 경계 (벽) 그리기 - 사각형 형태의 월드 경계 박스(벽)
    ax.plot([0, WORLD_W, WORLD_W, 0, 0], [0, 0, WORLD_H, WORLD_H, 0], lw=2)

    # 장애물(원형)
    for (cx, cy, r) in OBSTACLES:
        circ = plt.Circle((cx, cy), r, edgecolor='tab:red', facecolor='none', lw=2)
        ax.add_patch(circ)    # 축에 원 패치를 추가 - 라이다 감지 대상 장애물(빨간색 원)

    # 에이전트 삼각형(방향 표시)
    # 에이전트(즉, LiDAR 센서가 달린 로봇 본체)를 삼각형 형태로 그려주는 코드
    x, y, th = agent["x"], agent["y"], agent["theta"]
    L = 0.6    # 삼각형의 길이 스케일(scale)
    tri = np.array([    # 삼각형 좌표 계산
        [ x + np.cos(th) * L, y + np.sin(th) * L ], # 앞쪽. 현재 방향(th)을 따라 L 거리 앞쪽 위치
        [ x + np.cos(th + 2.5) * L / 1.5, y + np.sin(th + 2.5) * L / 1.5],
        # 왼쪽 뒤. 본체 중심에서 방향을 왼쪽으로 2.5rad (~143°) 돌린 방향
        [ x + np.cos(th - 2.5) * L / 1.5, y + np.sin(th - 2.5) * L / 1.5],
        # 오른쪽 뒤. 본체 중심에서 오른쪽으로 2.5rad (~143°) 돌린 방향
    ])
    ax.fill(tri[:, 0], tri[:, 1], alpha=0.8, color='tab:blue', label='agent')

    # 레이 시각화 : plot_world() 함수에서 LiDAR 센서가 쏜 광선(ray)들을 시각적으로 표시.
    # 즉, 앞서 계산된 레이의 시작점(에이전트 위치)과 끝점(충돌 지점)을 선으로 그려주는 역할

```

```

if rays_endpoints is not None: # rays_endpoints는 각 레이의 시작점, 끝점 좌표를 담은 리스트
    for (x0, y0, x1, y1) in rays_endpoints:
        # 두 점을 연결하는 선으로 표시
        ax.plot([x0, x1], [y0, y1], lw=1, alpha=0.8)

ax.legend(loc='upper right')
plt.tight_layout()
plt.show()

```

```

if __name__ == "__main__":

```

```

    # obs : 각 레이(ray)가 감지한 충돌 거리 배열 (길이 = NUM_RAYS)   obs = [3.2, 4.7, 6.5, ...]   # 거리값
    # ang : 각 레이의 절대 각도(rad) 배열   ang = [0.0, 0.2, 0.4, ...]   # 각도값
    obs, ang = cast_lidar(agent["x"], agent["y"], agent["theta"])   # LiDAR 센서 스캔
    endpoints = []   # 시각화용 레이 끝점 좌표
    for d, a in zip(obs, ang):   # 각 레이에 대해
        # 각 레이별로 시작점(x0, y0) 과 충돌 지점(x1, y1) 좌표를 계산
        x0, y0 = agent["x"], agent["y"]   # 에이전트 위치. 레이 시작점 (에이전트 위치)
        x1, y1 = x0 + np.cos(a)*d, y0 + np.sin(a)*d   # 레이 끝점. 충돌 지점 좌표
        # np.cos(a)와 np.sin(a)로 "각도 → 방향 벡터" 변환 후, 거리 d만큼 곱해 끝점을 구함.
        # 결과적으로 endpoints는 이런 리스트가 된다.
        # [
        #     (3.0, 3.0, 8.0, 3.0),   # 0° 방향으로 5m
        #     (3.0, 3.0, 6.5, 4.2),   # 30° 방향으로 4.8m
        #     ...
        # ]   plot_world()에서 각 선(ray)을 그리는 데 쓰임
        endpoints.append((x0, y0, x1, y1))

```

```

print("LiDAR observation (distances):")

```

```

print(np.round(obs, 2))   # 거리값 출력
# [ 3.5   3.7   4.05  4.45  5.05  5.85  7.1   9.1  10.   10.   10.   2.5
#  2.25  2.2   2.2   2.25  2.45  9.95 10.   10.   10.   10.   10.   10.
#  10.   10.   10.   10.   10.   9.55  7.35  6.05]

```

```

plot_world(agent, endpoints)   # 월드 + 레이 시각화

```